crezicsaryan/Haryn-AI

Branch **main** Scanned **20 Oct 2026** Duration **15.8 s** Commits **23**

OVERALL HEALTH

84 / 100 Grade B

Weighted across eight categories: security, bugs, quality, testing, architecture, performance, dependencies and documentation.

0	41
CRITICAL	HIGH
76	**27**
MEDIUM	LOW

Most important findings

144 actionable of 147 total

- **Complex function** — CODE QUALITY
  `src/app/api/analyze/route.ts:29 · ×36 in 31 files · confirmed`
- **Potential N+1 query** — PERFORMANCE
  `scripts/verify-ai.js:47 · ×3 in 2 files · heuristic`
- **SQL built by string concatenation** — SECURITY
  `src/lib/graph/store.ts:250 · ×2 in 1 file · heuristic`
- **Long function** — CODE QUALITY
  `src/lib/ai/tokenize.ts:48 · ×32 in 25 files · heuristic`
- **Empty catch block swallows the error** — BUGS
  `src/lib/db.ts:156 · ×17 in 12 files · heuristic`

Tech stack

TypeScript	96.4%
JavaScript	2.5%
CSS	1.0%

Scan coverage

Files analysed	122
Directories	36
Source size	974 KB
Code chunks indexed	831

Recommendation

crezicsaryan/Haryn-AI — Paste a public GitHub repo URL, get an evidence-based codebase health report: security, bugs, dependencies, tests, architecture, commit activity — plus a retrieval-grounded AI agent that cites the source it answers from is a compact codebase primarily TypeScript (JavaScript, CSS), with 122 files across 36 directories. The overall Codebase Health Score is 84/100 (Good).

FIX BEFORE RELEASE No critical issues, but 41 high-severity findings should be fixed before the next release.

What is wrong, where, and why it matters

Top 5 distinct problems · ×N = one rule, many files

01 Complex function

HIGH · ×36 · CONFIRMED

`src/app/api/analyze/route.ts:29` · repeated in 31 files

WHAT IT IS

POST() is 107 lines with an estimated cyclomatic complexity of 20. High complexity correlates with defects and makes the function hard to test.

FIX

Extract each branch into a named function so the caller reads as a list of steps. Target under 10 decision points; the worst offenders here are usually a long `if/else` chain that is really a lookup table.

IMPACT

Difficult to reason about, test and safely change.

```
POST() · 107 lines · complexity ≈ 20
```

02 Potential N+1 query

HIGH · ×3 · 60% HEURISTIC

`scripts/verify-ai.js:47` · repeated in 2 files

WHAT IT IS

A database or network call appears to execute inside a loop. Each iteration issues its own round-trip, so the cost grows linearly with the number of records.

FIX

Batch the required records into a single query (e.g. an IN clause or a join) and process the result in memory.

IMPACT

Response time degrades sharply as data grows, and the database sees a burst of small queries.

```
loop at scripts/verify-ai.js:47 → call at :50 · .filter(Boolean)
```

03 SQL built by string concatenation

HIGH · ×2 · 50% HEURISTIC

`src/lib/graph/store.ts:250` · repeated in 1 file

WHAT IT IS

Query strings assembled from variables can be manipulated to alter query semantics.

FIX

Use parameterised queries / prepared statements instead of building SQL by concatenation.

IMPACT

SQL injection if the interpolated value is user-controlled.

```
db.prepare(`SELECT COUNT(*) AS c FROM graph_nodes WHERE ${whereSql}`).get(...params)?.c
```

04 Long function

MEDIUM · ×32 · 65% HEURISTIC

`src/lib/ai/tokenize.ts:48` · repeated in 25 files

WHAT IT IS

tokenize() is 26 lines with an estimated complexity of 12.

FIX

Split at the seams: every block with its own inputs and output becomes a function. Keep the original as a short orchestrator.

IMPACT

Harder to unit-test in isolation.

```
tokenize() · 26 lines · complexity ≈ 12
```

05 Empty catch block swallows the error

MEDIUM · ×17 · 85% HEURISTIC

`src/lib/db.ts:156` · repeated in 12 files

WHAT IT IS

An exception is caught and discarded with no handling, logging or rethrow.

FIX

Log or rethrow: `catch (err) { logger.error(err); throw err; }`. Only swallow it if the failure is genuinely irrelevant, and say why in a comment.

IMPACT

Failures become invisible with no diagnostic trail.

```
} catch {
```

Verification status

What this report can and cannot claim

144	42	102	0
DETECTED	CONFIRMED IN SOURCE	HEURISTIC — VERIFY	FIXES VERIFIED

Every finding above was **detected** by a static rule reading the source. The repairs are **suggestions** — this analysis reads code, it never changes it, so **0 fixes are applied and 0 are verified**. A finding marked *heuristic* is a pattern to check, not a proven defect.

Proposed fixes

one decision each

01 **Complex function — 36× across 31 files**
Extract each branch into a named function so the caller reads as a list of steps. Target under 10 decision points; the worst offenders here are usually a long `if/else` chain that is really a lookup table.

02 **Long function — 32× across 25 files**
Split at the seams: every block with its own inputs and output becomes a function. Keep the original as a short orchestrator.

03 **Large file — 20× across 20 files**
Look for two responsibilities sharing a file and separate them. If it is one cohesive module, this is informational rather than a defect.

Tests

evidence, not results

Test suite executed	No — not run
Test files / source files	18 / 95
Frameworks detected	none

Dependencies

Declared	16
Vulnerable	0
Outdated	8
Healthy	8

Knowledge graph

Nodes / edges	756 / 2,024
Files parsed into the graph	113
Skipped as unsupported	0
Graph completeness	complete

Next steps

in this order

01 **Resolve the 41 critical and high findings first**
Start with sql built by string concatenation in src/lib/graph/store.ts. These are the ones worth delaying a release for.

02 **Collapse the repeated patterns — 3 rules, 88 findings**
One decision per rule removes 88 findings. This is the cheapest work in the report per edit.

03 **Schedule the 103 medium and low findings**
None of these block a release. Group them by area rather than fixing them one at a time.

04 **Re-run the analysis to confirm the fixes**
This report cannot verify a repair: it read the code, it did not change it. Re-analysing after the fix is what turns a suggestion into a verified result.

Limitations

- No test suite was executed. The counts above describe test *files*, not passing or failing tests.
- 102 findings are heuristic — the rule reports a pattern, not a proven runtime defect.
- Static analysis reads code; it does not run it. Behaviour that depends on runtime state, configuration or data is out of scope.
- 122 files and 973 KB of source were parsed for this report.